D1: Design and Executable Proposal Path

Name: Bani Kaur Pahwa
Student Number:221250709
Deliverable: D1

Design Overview

ParkOps D1 implements a controlled proposal workflow that lets a student request a change to their parking space. The starting example has a booking B1 assigned to space A17. The version for the booking is 0 and the person requires accessibility.

When a student requests a different parking space, the system checks whether the booking request meets all the rules. If it does meet the rules, a pending proposal is created, which reflects that the request is awaiting an approval. Creating a proposal does not mean that the change has been approved, the booking stays in its original space, version remains 0 and occupancy is unchanged until the booking is approved.

Responsibilities

1. StudentApplication: It gets the current booking and available parking spaces, checks the request, creates a pending request and gives a protected snapshot of the booking details. It runs the whole proposal process.
2. Booking: Stores the current parking booking details like booking ID, parking space, duration, version number and accessibility needs.
3. Space: Stores information about each parking space and other details like whether it is open, occupied or accessible.
4. Proposal: Stores details of the requested parking change separately from the current booking. It includes details like proposal ID, bookingID, space ID, version number and pending status.
5. DomainRules: Checks that IDs and booking durations follow the required rules.
6. Fixture: Provides sample booking and parking-space data so the program and tests produce consistent results every time.

Alternative Design ConsIDered

An alternative that I considered was to change the student's parking space as soon as a new space was requested. This would reduce steps in the implementation but we would have to combine checking the request, approval and changing the space into one step.

The design that I went through with created a pending proposal instead. This keeps original booking unchanged while the request awaits an approval. When the request is approved later, the system checks the rules for the second time before making the change. This design also makes it easier to check the rejected requests because the booking and the parking space occupant request should remain unchanged.

Informal Operation Contracts

Contract 1: Proposing a parking space change

The propose(targetId) operation creates a pending proposal if the requested parking space has a valid ID, exists, is open, is not occupied and is different from the student's current space. The proposal gets an ID generated by the application and stores the booking ID, requested space ID, booking version, policy version and PENDING status. Creating a proposal doesn't mean that the change has been approved or completed. In the given example, booking B1 must stay assigned to A17, its version must remain 0 and parking-space occupancy must not change.

Contract 2: Rejecting an invalid parking space request

The propose(targetId) operation rejects requests if the ID is invalid, the space does not exist, the space is occupied, the space is closed or the student requests their current space. The application reports an appropriate error and does not create a proposal for the rejected request. The booking, its version and the parking space occupancy must all remain unchanged.

This ensures that a failed request does not change any part of the system accidentally.

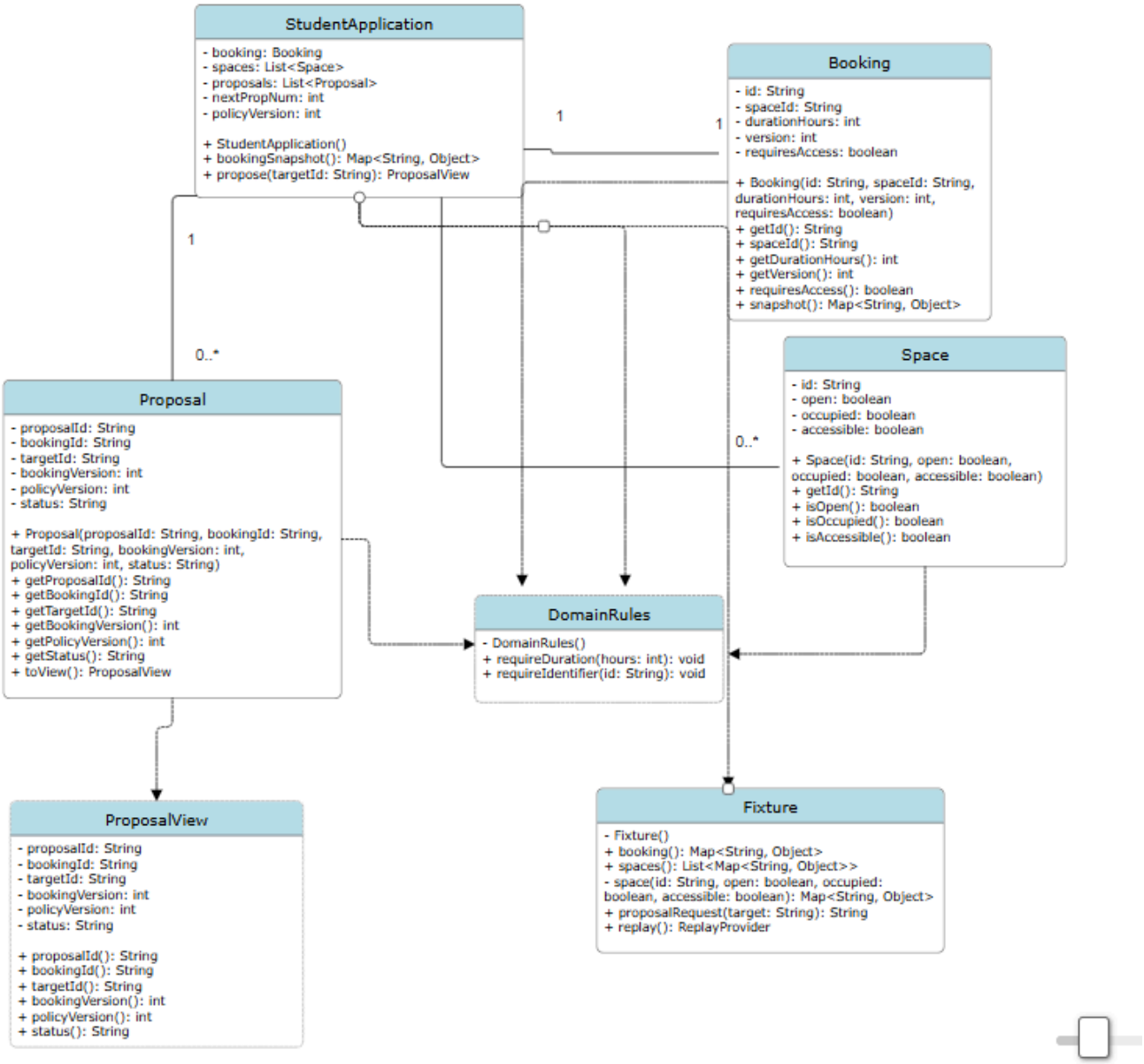
Limitations

The biggest limitation of the current model of the parking system is that the D1 code does not check whether a parking space meets a student's accessibility needs. Booking B1 requires an accessible space but B12 is not accessible. This means that the code can still create a pending proposal for B12 even when it does not meet the requirement. Therefore, the implementation does not check all parking space eligibility rules.
This system uses sample data stored in memory and cannot handle multiple requests at the same time.

Outputs

The outputs of the project are in the d1_output.txt file in the results/d1_output.txt

UML DIAGRAM


StudentApplication
- booking: Booking
- spaces: List<Space>
- proposals: List<Proposal>
- nextPropNum: int
- policyVersion: int
+ StudentApplication()
+ bookingSnapshot(): Map<String, Object>
+ propose(targetId: String): ProposalView
Booking
- id: String
- spaceId: String
- durationHours: int
- version: int
- requiresAccess: boolean
+ Booking(id: String, spaceId: String, durationHours: int, version: int, requiresAccess: boolean)
+ getId(): String
+ spaceId(): String
+ getDurationHours(): int
+ getVersion(): int
+ requiresAccess(): boolean
+ snapshot(): Map<String, Object>
Space
- id: String
- open: boolean
- occupied: boolean
- accessible: boolean
+ Space(id: String, open: boolean, occupied: boolean, accessible: boolean)
+ getId(): String
+ isOpen(): boolean
+ isOccupied(): boolean
+ isAccessible(): boolean
Proposal
- proposalId: String
- bookingId: String
- targetId: String
- bookingVersion: int
- policyVersion: int
- status: String
+ Proposal(proposalId: String, bookingId: String, targetId: String, bookingVersion: int, policyVersion: int, status: String)
+ getProposalId(): String
+ getBookingId(): String
+ getTargetId(): String
+ getBookingVersion(): int
+ getPolicyVersion(): int
+ getStatus(): String
+ toView(): ProposalView
DomainRules
- DomainRules()
+ requireDuration(hours: int): void
+ requireIdentifier(id: String): void
ProposalView
- proposalId: String
- bookingId: String
- targetId: String
- bookingVersion: int
- policyVersion: int
- status: String
+ proposalId(): String
+ bookingId(): String
+ targetId(): String
+ bookingVersion(): int
+ policyVersion(): int
+ status(): String
Fixture
- Fixture()
+ booking(): Map<String, Object>
+ spaces(): List<Map<String, Object>>
- space(id: String, open: boolean, occupied: boolean, accessible: boolean): Map<String, Object>
+ proposalRequest(target: String): String
+ replay(): ReplayProvider
1
1
1
0..*
0..*

Sequence Diagram

